Conceptual Progress Report

Stefanie Moreno

2026-03-21

Conceptual Progress Report

1. Recap of Project

Research Question: How much Markov memory (order k) is required for a Markov chain to reliably distinguish among biological sequence classes such as coding regions, noncoding regions, promoters, repetitive elements, and indels?

Algorithm and algorithm class: This project employs the k-th order Markov chain, one model per biological class. Each model estimates transition probabilities for all kmers and computes log-likelihoods for test sequences. Classification is performed by maximum likelihood across models.

Data: Labeled DNA sequences from Ensembl/UCSC (coding/noncoding), EPD (promoters) RepeatMasker (repeats), and simulated or annotated indel regions will be used for training. Unlabeled test sequences will be generated from a data simulation app to evaluate classification performance.

2. Inputs, Outputs, and Assumptions

2.1 Inputs:

Training sequences

- Format: FASTA or plain text
- Alphabet: A, C, G, T
- Labeled by class: {coding, noncoding, promoter, repeat, indel}

Hyperparameters:

- Markov order k (integer, 1-6 depending on class)
- Smoothing constant -> (default = 1 for Laplace smoothing)

Test sequences

- Format: FASTA or plain text
- Unlabeled DNA sequences generated from a data simulation app

2.2 Outputs:

Trained Markov models for each class and each k, containing:

- Dictionary mapping prefix (kmer prefix) -> counts of next bases
- Normalized transition probabilities

For each test sequence:

- Log-likelihood under each class model
- Predicted class (argmax)
- Accuracy curves vs. k
- Confusion matrices
- Per-class likelihood distributions

2.3 Assumptions:

- All sequences are uppercase A/C/G/T and if not, will be accounted for within code (ignored) to prevent triggering errors
- Training sequences for each class sufficiently long to estimate 4^k states (see table A)- consider using Burrows-Wheeler Transform with suffix array to compress long sequences for computation for higher order k (Pokzrywa, 2009)
- Transition probabilities for each prefix sum to 1 after smoothing
- Log-likelihood is normalized by sequence length to ensure comparability
- Models for different classes trained independently

k	4^k States	Minimum bp (10×)	Better bp (20×)	Safe Range
1	4	40 bp	80 bp	100 bp
2	16	160 bp	320 bp	1,000 bp
3	64	640 bp	1,280 bp	10,000 bp
4	256	2,560 bp	5,120 bp	50k–100k bp
5	1,024	10,240 bp	20,480 bp	200k–500k bp
6	4,096	40,960 bp	81,920 bp	1–5 million bp

(Table A. Minimum sequence length thresholds for reliable estimation of k-order Markov models, based on the number of states (4^k) and recommended minimum counts per state (10× for minimum, 20× for better performance). The safe range indicates the typical sequence lengths needed to ensure robust model training without overfitting or underfitting due to sparse data.)

3. Pseudocode

3.1. Data structures used:

- Valid bases: {A, T, G, C}
- counts[prefix][base] -> integer
- probs[prefix][base] -> float

3.2 Modules and Functions:

Implementation Script: markov_model.py All implementation functions are contained in a single Python script, src/markov_model.py, which serves as the main implementation module for the project.

Module 1: Input Handling

```
read_fasta()
load_class_seqs()
```

Module 2: Core Markov Model

```
count_kmers()
normalize_counts()
train_markov_model()
log_likelihood()
classify_sequence()
train_all_models()
classify_test_set()
```

Module 3: Reporting

```
compute_summary_stats()
write_output_files()
generate_graphs()
```

Module 4: Evaluation

```
Evaluate_model_performance()
```

3.3 Supporting Project Directories

Source Code Directory The source code directory contains the implementation script and additional helper scripts used for command-line execution. Only markov_model.py defines the functions listed above in 3.2. The other scripts are wrappers and utilities that support execution but do not contain core conceptual modules.

src/

```
markov_model.py -> implementation script (conceptual modules)
classify.py -> command-line wrapper
train_models.py -> command-line wrapper
utils.py -> helper functions
__init__.py -> package marker
```

Test Suite The tests directory contains pytest files used to validate the correctness of the implementation. These tests are not part of the program's conceptual modules; they are external validation artifacts.

tests/

```
test_kmer_counting.py
test_transition_probs.py
test_log_likelihood.py
test_markov.py
test_markov_k2.py
test_end_to_end.py
conftest.py
```

Input Files This directory contains prototype and real FASTA datasets used as input to the program. These files are external resources and not part of the program's modular design.

data/

```
data/prototype/
data/real/
```

Output Files The results/ directory is the designated output location for all generated files, including classification results, summary statistics, and visualizations. This directory is external to the program's modular design and serves as a repository for outputs.

results/

```
accuracy_curves/
confusion_matrices/
likelihood_distributions/
logs/
per_sequence_results.csv
summary_stats.txt
```

Documentation Suite The documentation directory contains the proposal, conceptual progress report, exploratory notes, README, requirements, references, gitignore, and any additional documents. It is external to the program's modular design and serves as a repository for project research, notes, and instructions.

docs/

```
Proposal.md
Conceptual_Progress_Report.Rmd
README.md
.requirements.txt
.gitignore
```

Auxillary Files The auxillary files include the project license and the pytest.ini file, required to run the test suite. These files are essential for project management and testing but are not part of the program's conceptual modules.

root/

```
LICENSE
pytest.ini
```

3.4 Pseudocode of functions for main script: markov_model.py

FUNCTION:

Read Fasta -> reads fasta file and extracts sequences with associated ids into a dictionary

```
def read_fasta(filepath):
    # filepath: string path to FASTA file
    initialize sequences as empty dictionary
    initialize current_header as none
    initialize current_seq as ""

    open file at filepath for reading:
    for each line in file:
        strip whitespace from line

    if line starts with ">":
        if current_header is not none:
            seq_dict[current_header] = current_seq
            current_header = line[1:]  # drop ">"
            current_seq = ""
            continue

    append line to current_seq
    if current_header is not none:
        seq_dict[current_header] = current_seq

    return seq_dict
```

Edge cases handled:

- Empty file returns {}
- Multi-line sequences
- Missing final newline

FUNCTION:

Load Sequences by Class -> loads multiple fasta files and converts them into a dictionary mapping each specialized sequence element (class) to a list of associated sequences

```
def load_class_seqs(class_fasta_paths):
    # initialize training_data as empty dictionary
    class_fasta_paths: {class_label -> filepath}

    for class_label in class_fasta_paths:
        filepath = class_fasta_paths[class_label]
        seq_dict = read_fasta_to_dict(filepath)
        seq_list = list of seq_dict[header] for each header in seq_dict
        training_data[class_label] = seq_list

    return training_data
```

Edge cases covered:

- Missing file -> i/o error
- Empty fasta -> class gets empty list
- Headers irrelevant -> safely discarded

FUNCTION:

Count kmers -> scans sequences to count times each kmer prefix is followed by each of the four nucleotides

```
def count_kmers(sequences, k):
    initialize counts as empty dictionary {}

    for each seq in sequences:
        clean seq by removing all characters not in (A, C, G, T)

    if length(seq) <= k:
        continue

    for i from 0 to length(seq) - k - 1:
        prefix = seq[i: i+k]

    if prefix not in counts:
        counts[prefix] = {A:0, C:0, G:0, T:0}
        increment counts[prefix][next_base] by 1

    return counts
```

Edge cases covered:

- Empty sequence list returns {}
- Sequences shorter than k discarded
- Non-ATGC characters ignored

FUNCTION:

Convert Counts to Probabilities (LaPlace smoothing) -> converts raw kmer counts to normalized transition probabilities with LaPlace smoothing

```
def normalize_counts(counts, alpha):
    initialize probs as empty dictionary {}

    for prefix in counts:
        total = sum(counts[prefix][b] + alpha for b in {A, T, G, C})
        probs[prefix] = empty map

    for each base in {A, T, G, C}:
        smoothed = counts[prefix][base] = smoothed / total

    return probs
```

Edge cases covered:

- Empty counts returns empty probs
- Smoothing fixes prefixes with all zero counts

- Alpha <= 0 -> invalid

FUNCTION:

Train k-Order Markov Model -> builds a complete k-order Markov model for a class by counting kmers and normalizing to probabilities with smoothing

```
def train_markov(sequences, k, alpha):
    if sequences empty:
        raise ValueError("No training data provided")

    counts = count_kmers(sequences, k)
    probs = normalize_counts(counts, alpha)

    return probs
```

Edge case covered:

- Empty training set -> explicit error

FUNCTION:

Compute Log Likelihood -> computes normalized log likelihood of a sequence under trained Markov model

```
def log_likelihood(seq, probs, k):
    clean seq by removing all chars not in {A,C,G,T}  # clean test seqs
    logL = 0  # initialize log likelihood at zero

    if length(seq) <= k:
        return -inf  # returns undefined for mathematically undefined circumstance

    for i from 0 to length(seq) - k - 1:
        prefix = seq[i: i+k]
        next_base = seq[i+k]

    if prefix not in probs:
        p = 1 / (4^k)

    else:
        p = probs[prefix][next_base]
        logL = += log(p)

    return logL/length(seq)
```

Edge cases covered:

- Sequence shorter than k mathematically undefined
- Unseen prefix -> uniform fallback
- Non-ATGC characters removed
- Empty probs -> all prefixes unseen

FUNCTION:

Classify a Sequence Across Models -> scores a sequence under all class-specific models and returns the class with the highest resulting probability

```
def classify_sequence(seq, model_dict, k):
    best_class = none  # initialize to no prediction
    best_score = -inf  # initialize to worst case

    if model_dict empty:
        raise ValueError("No models available for classification")

    for class_label in model_dict:
        score = log_likelihood(seq, model_dict[class_label], k)

    if score > best_score:
        best_score = score
        best_class = class_label

    return best_class, best_score
```

Edge cases accounted for:

- All models return -inf -> best_class is whichever appears first, but outputs= -inf, which can be tracked in summary stats
- Empty model dict -> call error

FUNCTION:

Train All Models -> builds Markov models for each class and every k-value and stores in nested dictionary

```
def train_all_models(training_data, k_values, alpha):
    models = empty map  # initialize models dictionary

    if training_data empty:
        raise ValueError("No training data provided")

    for k in k_values:
        if k < 1 or k > 6:
            raise ValueError("k must be between 1 and 6")

        if alpha <= 0:
            raise ValueError("Alpha must be greater than 0")

        if length(seq) <= k:
        return -inf  # returns undefined for mathematically undefined circumstance

        models[k] = empty map

    for class_label in training_data:
        seqs = training_data[class_label]
        probs = train_markov_model(seqs, k, alpha)
        models[k][class_label] = probs

    return models
```

Edge cases covered:

- Training_data[class] empty -> explicit error
- All sequences too short for given k -> undefined, but will be tracked in summary stats
- Alpha <= 0 -> explicit error

FUNCTION:

Classify Test Set -> classifies each test sequence using the trained models for specified k and returns predicted classes and scores

```
def classify_test_set(test_sequences, model, k):
    if k not in models:
        raise ValueError("No models available for this k")

    if models[k] is empty:
        raise ValueError("No class models available for classification")

    results = empty list # initialize results list

    for each seq in test_sequences:
        predicted_class, score = classify_sequence(seq, models[k], k)
        append(seq, predicted_class, score) to results

    return results
```

Edge cases covered:

- k not trained -> explicit error
- Models[k] empty -> explicit error
- Test sequences empty -> return empty list

FUNCTION:

Compute Summary Statistics -> computes metrics overview from classification results for reporting

```
def compute_summary_stats(results):
    stats = empty map  # initialize stats dictionary

    stats["total"] = length(results)
    stats["unclassified"] = count entries where predicted_class is None
    stats["neg_inf"] = count entries where score == -inf
    class_counts = empty map  # initialize class counts dictionary

    for each (_, cls, _) in results:
        if class not in class_counts:
            class_counts[cls] = 0  # initialize count for new class
            class_counts[cls] += 1  # increment count for predicted class

    stats["class_counts"] = class_counts

    return stats
```

Edge cases covered:

- Results empty ->all stats = 0, class_counts empty
- All predictions None -> unclassified = total
- All scores −inf -> neg_inf = total
- Only one class predicted -> class_counts has one entry

FUNCTION:

Output Files -> writes classification results and summary stats to output files csv and txt for downstream analysis

```
def write_output_files(results):
    # Write per-sequence results
    with open ("per_sequence_results.csv") for writing:
        write "sequence,predicted_class,score\n"

    for each (seq, cls, score) in results:
        write seq + "," + cls + "," + str(score) + "\n"

    # Write summary statistics
    with open("summary_stats.txt") for writing:
        write "Total Sequences: " + str(stats["total"]) + "\n"
        write "Unclassified: " + str(stats["unclassified"]) + "\n"
        write "Negative Infinite Scores: " + str(stats["neg_inf"]) + "\n"
        write "Class Counts:\n"
    for each class_label in stats["class_counts"]:
        write "  " + class_label + ": " + str(stats["class_counts"][class_label]) + "\n"

return
```

Edge cases covered:

- Results empty -> writes empty csv and summary with zeros
- Sequences contain only A/C/T/G -> csv safe
- Extremely long sequences -> large file but still valid

FUNCTION:

Generate Graphs -> creates visualizations from classification results

** I haven't decided what types of graphs I will be generating yet. I like to see what my output data looks like before committing to a graphing format **

FUNCTION:

Evaluate Model Performance -> computes advanced evaluation metrics across all k values, including accuracy curves, confusion matrices, and per class likelihood distributions

```
def evaluate_model_performance(all_results, true_labels):
    # all_results: map k -> list of (seq, predicted_class, score, class_scores)
    # true_labels: map seq -> true_class
    metrics = empty map  # initialize metrics dictionary
    # accuracy vs k
    accuracy = empty map  # initialize accuracy dictionary

    for each k in all_results:
```

```
        correct = 0  # initialize correct count for this k
        total = length(all_results[k])  # total sequences classified for this k

    for each (seq, pred, _, _) in all_results[k]:
        if pred == true_labels[seq]:
            correct += 1  # increment correct count if prediction matches true label
            accuracy[k] = correct / total  # compute accuracy for this k
            metrics["accuracy_vs_k"] = accuracy  # store accuracy curve in metrics

    # confusion matrices
    confusion = empty map  # initialize confusion matrices dictionary

    for each k in all_results:
        matrix = int zero matrix of size (num_classes × num_classes)  # initialize confusion matrix for

    for each (seq, pred, _, _) in all_results[k]:
        true = true_labels[seq]  # get true label for this sequence

        if true not in matrix:
            raise ValueError("True label not found in confusion matrix classes")

        matrix[true][pred] += 1  # increment confusion matrix cell for true vs predicted class
        confusion[k] = matrix  # store confusion matrix for this k in metrics

    metrics["confusion_matrices"] = confusion  # store confusion matrices in metrics

    # per-class likelihood distributions
    likelihoods = empty map  # initialize likelihood distributions dictionary

    for each k in all_results: # iterate over each k value
        per_class = empty map  # initialize per-class likelihoods dictionary for this k

    for each (seq, pred, score, class_scores) in all_results[k]:
        if class_scores is None:
            raise ValueError("Class scores missing for sequence: " + seq)
        for each class_label in class_scores:
            # append score for this class to the list of scores for this class in per_class
            append class_scores[class_label] to per_class[class_label]

            likelihoods[k] = per_class  # store per-class likelihood distributions for this k in metrics
            metrics["likelihood_distributions"] = likelihoods  # store all evaluation metrics in the me

    return metrics
```

Edge cases covered:

- True labels missing entries -> explicit error
- All predictions -inf -> accuracy = 0 for all k
- No results for a k -> skip that k
- Class scores missing for a sequence -> explicit error

4. Complexity and Bottlenecks

4.1 Time Complexity:

Time complexity must account for training and classification phases

Training Phase: Consists of counting kmers and normalizing counts for each class

- Let:
  - N = number of sequences in a class
  - L = average sequence length
  - k = Markov order
  - C = number of classes

Counting kmers:

Each sequence contributes:

```
(L-k) kmers
```

Total cost per class:

```
O(N*(L-k)) ~ O(NL)
```

Normalizing counts: The number of possible states is 4^k and each state has 4 outgoing transitions Worst case normalization cost:

```
dO(4^k)
```

Total training cost across all classes:

```
O(C * (NL + 4^k))
```

Classification Phase:

For each test sequence of length L, extracting each prefix and scoring transitions:

```
O(L-k) ~ O(L)
```

If there are C classes:

```
O(C*L)
```

Overall Time Complexity:

The overall time complexity of the training and classification phases combined is:

```
O(C * (NL + 4^k) + C*L) ~ O(C * (NL + 4^k))
```

- Training dominates when 4^k becomes large

- Classification dominates when the number of test sequences is large
- Since k values will not exceed 6, this project will be computationally manageable
- For higher order k, consider Burrows-Wheeler Transform with suffix array to compress long sequences and ease computational load (Hu & Chen, 2024)

4.2 Space Complexity:

Space complexity must account for model and input sequence storage, as well as generated kmers

Model Storage:

A k-order Markov model stores: - Up to 4^k states - Each with 4 outgoing probabilities

Space per class:

```
O(4^k)
```

Total space across all classes:

```
O(c*4^k)
```

Sequence Storage:

- FASTA dictionary + list of sequences:

```
O(NL)
```

4.3 Bottlenecks

Exponential State Growth:

- 4^k becomes prohibitive for k > 8 (Ost & Takahashi, 2023)
- This affects both memory and normalization time
- For this project, k will not exceed 6

Large Training Sets:

- If N and L are large (e.g., whole genome reads), counting k mers becomes the dominant cost -For this project, training sets will be limited to minimum size needed to estimate 4^k states- see table in assumptions section
  - Consider using Burrows-Wheeler Transform with suffix array to compress long sequences for computation for k order 5-6 (Pokzrywa, 2009)

Classification Across Many Classes:

- If C is large, classification cost scales linearly with the number of classes
  - In this project, C will be limited to 5: coding, noncoding, promoters, repetitive sequences, indels

4.4 Performance Considerations

- The complexity of this algorithm scales exponentially with k and linearly with N, L, and C. Performance becomes a problem when:
  - k too large -> model tables exceed memory
  - Very long sequences -> classification becomes slow
  - Many classes -> scoring each sequence becomes too costly
  - Highly imbalanced datasets -> some classes produce sparse models, increasing smoothing overhead.

4.5 Mitigation Strategies

a. Cap k at a biologically meaningful value

b. Sparse dictionaries: store only observed prefixes instead of all 4^k (Ost & Takahashi, 2023)

c. Prefix pruning: discard prefixes with extremely low counts

d. Streaming FASTA parsing: avoid loading all sequences into memory

e. Approximate methods:

- downsample long sequences
- skip low information regions
- use fixed window scoring

5. Validation and Testing Strategies

5.1 Synthetic Short Sequence Test Cases (small, deterministic)

Minimal sequences

- Input: sequences like "AAAAA" with k=2
- Expected Behavior:
  - Only prefix "AA" appears
  - Transition probabilities deterministic
  - Log likelihood easy to compute manually

Edge cases

- These tests verify correctness of kmer counting, smoothing, prefix extraction, log likelihood accumulation, and error handling
- Expected Behavior:
  - Sequence length k → return -inf
  - Sequence with invalid characters → cleaned before scoring
  - Empty class model → classification error raised

5.2 Synthetic & Real Stress Tests

Synthetic random DNA

- Generate random sequences of length 10k–100k
- Train with moderate k (3–6)
- Expected Behavior:
  - Uniform transition probabilities
  - Log likelihoods cluster around expected random sequence entropy

Real biological data

- Input: coding vs. non coding FASTA files
- Expected Behavior:
  - Coding sequences show stronger periodicity and codon structure
  - Non coding sequences show flatter distributions
  - Classifier should separate them above random chance
  - Log likelihoods should be higher for sequences similar to training data
  - Smoothing should prevent zero probabilities
  - Increasing k should initially improve discrimination, then degrade when 4^k becomes too large
- Evidence of Incorrect Behavior:
  - Negative infinite scores for sequences that should be valid
  - Probabilities not summing to 1 for any prefix
  - Identical sequences producing different scores across runs
  - Classifier predicting the same class for all sequences
  - Sudden drop in accuracy when increasing k slightly (indicates state explosion)

5.3 Automated Tests

Unit Tests

- Fasta parsing
- kmer counting
- Smoothing
- Prefix extraction
- Log likelihood for known sequences

Property Tests

- All probability rows sum to 1
- Increasing sequence length should not decrease log likelihood under identical models
- Removing invalid characters should not change valid transitions

End to End Tests

- Train on two simple classes, classify a small test set, verify expected predictions
- Train on real data, ensure no errors or empty models

6. Updated Pitfall and Risk Log

6.1 Still Relevant Risks from Part 1

State explosion for large k

Mitigation:

- cap k, use sparse dictionaries (Ost & Takahashi, 2023)
- implement Burrows-Wheeler transform with suffix array to compress sequences such that finding all occurrences of a motif is possible in O(m), where m = motif length (Pokrzywa, 2009)

Imbalanced training data

Mitigation:

- smoothing, minimum count thresholds

Invalid characters in sequences

Mitigation:

- cleaning step in both training and scoring

** Short sequences**

Mitigation:

- return -inf for sequences k

6.2 New Risks Identified During Pseudocode Development

Empty class models

Mitigation:

- explicit error in classify_sequence and classify_test_set

Smoothing errors

Mitigation:

- enforce alpha > 0

Silent failures in normalization

Mitigation:

- assert probability rows sum to 1

Incorrect prefix extraction

Mitigation:

- unit tests for prefix boundaries

Log underflow: Probability becomes so small can no longer be represented numerically, collapses to -inf or NaN. A k-order Markov model multiplies many conditional probabilities, log space helps but doesn't eliminate the problem

Mitigation:

- use log space accumulation exclusively

7. Generative AI Usage

7.1 Tool Used

`CoPilot`

7.2 Prompts Provided

- Is the following program pseudocode structured correctly? Also, identify edge cases accounted for in each function
- Explain in detail and with equations the time and space complexity of a k-order Markov model classifier for DNA sequences, including training and classification phases
- How does increasing k impact the computational complexity of this model?
- Break down the complexity of MCMC algorithm using the parameters N (number of sequences), L (sequence length), C (number of classes), and k (Markov order). Identify which terms dominate as these parameters grow and explain the implications for performance.
- How does the exponential state 4^k impact performance and at what k does it become too computationally taxing?
- What are the length thresholds for sequences to sufficiently test 4^k ?
- What mitigation techniques can be used to handle the state explosion problem for higher k values in Markov models?

7.3 Influence on Pseudocode

- Helped articulate edge cases
- Provided clarity on separation of training, scoring, reporting, and evaluation modules

7.4 Rationale for Use

- Ensure pseudocode is logically modular and accounts for edge cases
- Verify complexity analysis and testing strategies align with standard computational biology practices

REFERENCES

Hu, S. & Chen, C. (2024). Performance models for sequence alignment algorithms based on Burrows-Wheeler Transform. ICBBT (16th Annual): p.1-9. https://dl.acm.org/doi/10.1145/3674658.3674661

Ost, G. & Takahashi, D. L. (2023). Sparse Markov models for high dimensional inference. Cornell University AriXiv: 2202.08007, p.1-44. https://arxiv.org/abs/2202.08007

Pokrzywa, R. (2009) Application of the Burrows-Wheeler Transform for searching for tandem repeats in DNA sequences. Int J Bioinform Res App. 5(4):432-46. https://doi.org/10.1504/IJBRA.2009.027517

ADDITIONAL RESOURCES

Bi, C. (2009). A Monte Carlo EM algorithm for de novo motif discovery in biomolecular sequences. IEEE/ACM Transactions on Comp Biol and Bioinf, 6(3): 370-387. https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=4641908

Bi, C. (2009). DNA motif alignment by evolving a population of Markov chains. *BMC Bioinformatics.* 10 Suppl 1(Suppl 1):S13. https://doi.org/10.1186/1471-2105-10-S1-S13

Gelfond, J. A., Gupta, M., & Ibrahim, J. G. (2009). A Bayesian hidden Markov model for motif discovery through joint modeling of genomic sequence and ChIP-chip data. Biometrics, 65(4):1087-95. https://doi.org/10.1111/j.1541-0420.2008.01180.x

Gusfield, D. (1997). Algorithms on strings, trees, and sequences: Computer science and computational biology. Cambridge University Press. https://www.inf.ufes.br/~berilhes/Cursos/TBO2008-1/gusfield.pdf

Hendrix, D. A. (2019). Applied bioinformatics of nucleic acid sequences. Creative Commons. Pressbooks. https://open.oregonstate.education/appliedbioinformatics/front-matter/applied-bioinformatics/#front-matter-490-section-1

Huang, W., Umbach, D. M., Ohler, U., & Li, L. (2006). Optimized mixed Markov models for motif identification. BMC Bioinformatics, 7(279). https://doi.org/10.1186/1471-2105-7-279

Ma, Y., Chen, H., Kang, J., Guo, X., Sun, C., Xu, J., Tao, J., Wei, S., Dong, Y., Tian, H.,Lv, W., Jia, Z., Bi, S., Shang, Z., Zhang, C., Lv, H., Jiang, Y., Zhang, M. (2026). The hidden Markov model and its applications in bioinformatics analysis. Genes & Diseases,13(1):101729. https://doi.org/10.1016/j.gendis.2025.101729

Maiti, A. & Mukherjee, A. (2015). On the Monte Carlo expectation maximization for finding motifs in DNA sequences. IEEE Journal of Biomedical and Health Informatics, 19(2): 677-687. https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=6812135

Pokrzywa, R. (2008). Searching for unique DNA sequences with the Burrows-Wheeler Transform. Biocybernetics and Biomedical Engineering, 28(1): 95-104. https://ibib.pl/images/ibib/grupy/Wydawnictwa-Tomy/dokumenty/2008/BBE_28_1_095_FT.pdf

Powell, M. (2001). Compressed-domain pattern matching with the Burrows-Wheeler Transform. Honours Project Report, p.1-51. https://corpus.canterbury.ac.nz/research/powell2001_hons_report.pdf

Schbath, S., Prum, B., & De Turchkheim, E. (1995). Exceptional motifs in different Markov chain models for a statistical analysis of DNA sequences. Journal of Computational Biology, 2(3). https://doi.org/10.1089/cmb.1995.2.417

Schutz, F. & Delorenzi, M. (2008). MAMOT: Hidden Markov modeling tool. Bioinformatics, 24(11): 1399-1400. https://doi.org/10.1093/bioinformatics/btn201

Toivonen, J., Das, P. K., Taipale, J., & Ukkonen, E. (2020). MODER2: first-order Markov modeling and discovery of monomeric and dimeric binding motifs, Bioinformatics, 36(9): 2690–2696, https://doi.org/10.1093/bioinformatics/btaa045

Vanet, A., Marsan, L., & Sagot, M. F. (1999). Promoter sequences and algorithmical methods for identifying them. Research in Microbiology, 150(9-10): 779-799. https://doi.org/10.1016/S0923-2508(99)00115-1

Won, K. J., Sandelin, A., Marstrand, T. T., & Krogh, A. (2008). Modeling promoter grammars with evolving hidden Markov models. Bioinformatics, 24(15): 1669-1675. https://doi.org/10.1093/bioinformatics/btn254

Woodard, D. B. & Rosenthal, J. S. (2012). Convergence rate of Markov chain methods for genomic motif discovery. Annals of Statistics, 1-36. https://www.probability.ca/jeff/ftpdir/motifFinding.pdf

Yada, T., Totoki, Y., Ishikawa, M., Asai, K., & Nakai, K. (1998). Automatic extraction of motifs represented in the hidden Markov model from a number of DNA sequences. Bioinformatics, 14(4): 317-325. https://doi.org/10.1093/bioinformatics/14.4.317

Zhao, X., Huang, H., & Speed, T. P. (2004). Finding short DNA motifs using permuted Markov models. RECOMB '04: Proceedings of 8th Annual Int'l Conf. Res. Comp MB, p.68-75. https://doi.org/10.1145/974614.974624

Zhao, X. & Sze, S. H. (2011). Motif finding in DNA sequences based on skipping nonconserved positions in background Markov chains. Journal of Computational Biology, 18(5). https://doi.org/10.1089/cmb.2010.0197